URL → GIF

Website Screenshot to Animated GIF Generator

Technical Architecture & Implementation Guide

Version 1.0 • March 2026

Confidential — For Development Team Only

Table of Contents

1. Project Overview
2. Frontend Design Guide
3. Backend Architecture
4. Website Crawling & Screenshot Pipeline
5. GIF Generation Pipeline
6. Security & Abuse Prevention
7. Infrastructure & Deployment
8. API Reference
9. Monitoring & Observability

1. Project Overview

This guide details the architecture for a web application that lets users paste a website URL and receive an animated GIF showcasing that website. The GIF is designed for embedding in portfolios, case studies, README files, and social media showcases.

1.1 Core User Flow

1. **Step 1:** User pastes a URL into the input field.
2. **Step 2:** System validates the URL, checks rate limits, and enqueues a screenshot job.
3. **Step 3:** A headless browser navigates to the URL, captures multiple viewport-scrolled screenshots.
4. **Step 4:** Screenshots are stitched into an animated GIF with smooth scroll animation.
5. **Step 5:** User previews the GIF and downloads it, or copies an embed link.

1.2 Key Goals

- Fast turnaround: under 30 seconds for typical websites.
- Professional-quality GIFs at multiple resolution options.
- Robust security to prevent abuse, SSRF attacks, and resource exhaustion.
- Scalable architecture supporting 1,000+ concurrent users.

1.3 Tech Stack Summary

Layer	Technology	Rationale
Frontend	Next.js 15 + React 19	SSR, API routes, App Router
Styling	Tailwind CSS v4 + Framer Motion	Utility-first, smooth animations
Backend API	Next.js API Routes / Node.js	Unified deployment, serverless-ready
Job Queue	BullMQ + Redis	Reliable async job processing
Browser Engine	Playwright (Chromium)	Best headless browser automation
GIF Encoding	FFmpeg + gifski	High-quality, optimized GIF output
Storage	AWS S3 / Cloudflare R2	CDN-backed asset delivery
Database	PostgreSQL + Prisma	Job tracking, user analytics, rate limits
Cache	Redis	Rate limiting, session data, job status

Auth (optional)	NextAuth.js / Clerk	Optional user accounts for history

2. Frontend Design Guide

The frontend should feel like a premium developer tool — fast, focused, and visually refined. The aesthetic direction is “modern utility with a creative edge”: clean dark surfaces, monospace accents, and satisfying micro-interactions.

2.1 Design System

Color Palette

Token	Value	Usage
--bg-primary	#0A0A0B	Main background, dark surface
--bg-secondary	#141416	Card backgrounds, panels
--bg-elevated	#1C1C1F	Hover states, active cards
--accent-primary	#3B82F6	Primary CTA, links, focus rings
--accent-glow	#60A5FA	Subtle glow effects, highlights
--text-primary	#FAFAFA	Headings, primary text
--text-secondary	#A1A1AA	Body text, descriptions
--text-muted	#52525B	Placeholders, disabled
--border-subtle	#27272A	Card borders, dividers
--success	#22C55E	Completion states
--error	#EF4444	Error states, validation

Typography

- Display/Headings: “Space Mono” or “JetBrains Mono” — monospace gives a technical, developer-friendly feel.
- Body text: “Satoshi” or “General Sans” — modern geometric sans with excellent readability.
- Code/URL display: “Fira Code” — with ligatures enabled for the URL input area.

2.2 Page Layout & Components

Hero / Landing Section

A single-screen hero that immediately communicates the product value. The URL input should be the unmistakable focal point.

- Large headline: e.g., “Turn any website into a GIF” in 48–56px monospace.
- Subtitle: one line explaining the use case, 18–20px secondary text.

- The URL input bar: a wide, pill-shaped input with a gradient border glow on focus. Minimum 600px wide on desktop.
- A single CTA button: “Generate GIF →” with a shimmer/pulse animation on idle.
- Below the fold: 3–4 example GIFs auto-playing in a staggered grid to showcase output quality.

URL Input Component

This is the most critical UI element. Design it to feel premium and responsive:

- Dark input field (#141416) with a 1px border (#27272A) that transitions to a glowing blue border on focus.
- Real-time URL validation: green checkmark icon when valid, red shake animation when invalid.
- Paste detection: auto-trim whitespace, strip tracking parameters, show the cleaned URL.
- Protocol auto-prepend: if user types “example.com”, silently prepend “https://”.
- Loading state: the input border becomes an animated gradient bar (blue → purple → blue cycling).

Progress / Processing View

After submission, transition the hero into a progress dashboard:

- Step indicator showing: Validating → Loading page → Capturing screenshots → Generating GIF → Done.
- Each step has a spinner that becomes a checkmark on completion.
- Live preview: show captured screenshots as thumbnails appearing one by one in a film-strip row.
- Estimated time remaining displayed below the progress bar.
- A subtle particle or constellation animation in the background during processing.

Result / Download View

When the GIF is ready, present it with maximum impact:

- The GIF auto-plays in a device-frame mockup (laptop, phone, or tablet — user-selectable).
- Download button: primary CTA, prominent, with file size displayed.
- Copy embed link: secondary action, copies an <img> tag or markdown to clipboard.
- Customization panel (collapsible): adjust speed, resolution, frame count, loop behavior.
- Share buttons: direct share to Twitter/X, copy link for embedding.
- “Generate another” button to reset the flow.

2.3 Responsive Breakpoints

Breakpoint	Width	Layout Changes

Mobile	< 640px	Full-width input, stacked layout, smaller GIF preview
Tablet	640–1024px	Centered input, 2-column example grid
Desktop	1024–1440px	Wide input bar, 3-column grid, side panel for options
Wide	> 1440px	Max-width container (1200px), generous whitespace

2.4 Animation & Micro-interactions

- Page load: staggered fade-up for hero elements (headline → subtitle → input → examples), 100ms delay between each.
- Input focus: border glow expands outward with a 0.3s ease-out transition.
- Button hover: subtle scale(1.02) + shadow increase.
- Progress steps: slide-in from left with a spring easing curve.
- GIF reveal: scale from 0.95 to 1.0 with a fade, the device frame slides in from below.
- Error shake: horizontal translateX oscillation, 3 cycles over 0.4s.

2.5 Error & Edge Case UI

Scenario	UI Behavior
Invalid URL format	Red border + inline error: “Please enter a valid URL”
Site unreachable / timeout	Error card with retry button: “The site didn’t respond. Try again?”
Rate limit exceeded	Toast notification: “You’ve hit the limit. Wait X minutes.” + countdown
Blocked domain	Inline warning: “This domain is not supported for security reasons.”
GIF generation failure	Error card with option to retry or report the issue
Very long page	Info toast: “Large page detected. GIF will capture the first 5 scrolls.”

3. Backend Architecture

The backend follows an async job-processing pattern. The API accepts requests, validates them, and pushes work to a queue. Worker processes handle the heavy lifting (browser automation, GIF encoding) independently.

3.1 High-Level Architecture Diagram

```
[Client] → [API Gateway / Next.js API Routes]
    → [Validation & Rate Limiter (Redis)]
    → [Job Queue (BullMQ / Redis)]
    → [Worker Pool]
        → [Playwright: Screenshot Capture]
        → [FFmpeg/gifski: GIF Encoding]
        → [S3/R2: Asset Storage]
    → [Client polls /api/status/:jobId]
    → [Result returned with CDN URL]
```

3.2 API Endpoints

Method	Endpoint	Purpose
POST	/api/generate	Submit a URL for GIF generation
GET	/api/status/:jobId	Poll job progress (SSE or polling)
GET	/api/result/:jobId	Fetch the final GIF URL + metadata
GET	/api/health	Health check for load balancers
POST	/api/feedback	Optional: report bad results

3.3 POST /api/generate — Request Flow

1. **1. Input Validation:** Verify URL format (regex + URL constructor parse). Reject non-HTTP(S) schemes.
2. **2. Domain Security Check:** Resolve DNS, block private IPs (SSRF prevention), check against domain blocklist.
3. **3. Rate Limit Check:** Token-bucket algorithm via Redis. Keyed on IP + fingerprint.
4. **4. Deduplication:** Hash the normalized URL. If a recent job for the same URL exists, return cached result.
5. **5. Job Creation:** Insert job into PostgreSQL (status: PENDING), enqueue in BullMQ.
6. **6. Response:** Return 202 Accepted with { jobId, estimatedTime }.

3.4 Worker Process

Workers are stateless Node.js processes that consume jobs from BullMQ. Each worker:

- Launches a Playwright Chromium instance (pooled, not per-request).
- Navigates to the target URL with a 15-second timeout.
- Captures viewport screenshots at defined scroll intervals.
- Passes frames to the GIF encoder pipeline.
- Uploads the result to S3/R2 and updates the job status in PostgreSQL.
- On failure: retries up to 3 times with exponential backoff, then marks job as FAILED.

3.5 Database Schema (PostgreSQL)

```
CREATE TABLE jobs (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  url         TEXT NOT NULL,
  url_hash    VARCHAR(64) NOT NULL,
  status      VARCHAR(20) DEFAULT 'PENDING',
  progress    JSONB DEFAULT '{}',
  result_url  TEXT,
  file_size   INTEGER,
  frame_count INTEGER,
  ip_address  INET NOT NULL,
  user_agent  TEXT,
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  updated_at  TIMESTAMPTZ DEFAULT NOW(),
  expires_at  TIMESTAMPTZ DEFAULT NOW() + INTERVAL '24 hours'
);
```

```
CREATE INDEX idx_jobs_url_hash ON jobs(url_hash);
CREATE INDEX idx_jobs_status ON jobs(status);
CREATE INDEX idx_jobs_ip ON jobs(ip_address);
CREATE INDEX idx_jobs_expires ON jobs(expires_at);
```

> *GIF assets are auto-deleted after 24 hours via a scheduled cleanup job. This reduces storage costs and limits liability from cached third-party content.*

4. Website Crawling & Screenshot Pipeline

The crawling layer is the most security-sensitive and resource-intensive part of the system. It uses Playwright with Chromium in a locked-down configuration.

4.1 Playwright Configuration

```
const browser = await chromium.launch({
  headless: true,
  args: [
    '--no-sandbox',
    '--disable-setuid-sandbox',
    '--disable-dev-shm-usage',
    '--disable-gpu',
    '--disable-extensions',
    '--disable-background-networking',
    '--disable-default-apps',
    '--disable-sync',
    '--disable-translate',
    '--no-first-run',
    '--disable-web-security=false',
  ],
});
```

4.2 Screenshot Capture Strategy

The goal is to produce a smooth scrolling animation. The system captures a sequence of viewport-sized screenshots at calculated scroll positions.

Capture Parameters

Parameter	Default	Range	Notes
Viewport width	1280px	375–1920px	Selectable: mobile/tablet/desktop
Viewport height	800px	667–1080px	Matches viewport preset
Scroll step	400px	200–600px	Half-viewport for smooth motion
Max scroll depth	4000px	—	Prevents infinite-scroll abuse
Wait after scroll	300ms	200–500ms	Allows lazy content to load
Max frames	15	5–25	Controls GIF size

Page load timeout	15s	—	Hard cutoff

Capture Algorithm

1. **1.** Navigate to URL with networkidle or 15s timeout (whichever comes first).
2. **2.** Dismiss common overlays: cookie banners, newsletter popups (via common CSS selectors).
3. **3.** Measure document.scrollHeight to determine total scrollable area.
4. **4.** Calculate scroll positions: [0, step, step*2, ... min(maxDepth, scrollHeight)].
5. **5.** At each position: window.scrollTo({ top: pos, behavior: 'instant' }), wait 300ms, screenshot.
6. **6.** Capture a final frame paused at the top (scroll back to 0) for a clean loop.
7. **7.** Store all frames as PNG buffers in memory (not disk) for security.

4.3 Overlay Dismissal

Common modal/banner selectors to auto-dismiss before capture:

```
const DISMISS_SELECTORS = [
  '[class*="cookie"] button[class*="accept"]',
  '[id*="cookie"] button[class*="accept"]',
  '[class*="consent"] button[class*="agree"]',
  '[class*="popup"] button[class*="close"]',
  '[class*="modal"] button[aria-label="Close"]',
  '[class*="newsletter"] button[class*="close"]',
  '.onetrust-accept-btn-handler',
];
```

Each selector is attempted with a 1-second timeout. Failures are silently ignored — not all sites have these elements.

4.4 Content Type Handling

Content Type	Behavior
text/html	Standard screenshot flow
application/pdf	Reject with user-friendly message
image/*	Reject — suggest direct download instead
Redirect chain > 5	Abort and report error
HTTP 4xx / 5xx	Report “Site returned an error”
Login walls detected	Capture what’s visible, warn user

5. GIF Generation Pipeline

Raw screenshots need to be transformed into a smooth, optimized animated GIF. The pipeline uses FFmpeg for frame preparation and gifski for high-quality GIF encoding.

5.1 Pipeline Steps

1. **1. Frame Preparation:** Resize all PNG frames to the target output resolution (e.g., 800x500 for standard, 1200x750 for high-res). Use sharp or FFmpeg for resizing with Lanczos resampling.
2. **2. Frame Interpolation (optional):** For smoother scrolling, generate intermediate frames using FFmpeg's minterpolate filter. This doubles the frame count but produces butter-smooth motion.
3. **3. GIF Encoding:** Use gifski (Rust-based encoder) for best color quality at small file sizes. Falls back to FFmpeg's palettegen/paletteuse for environments without gifski.
4. **4. Optimization:** Run gifsicle for lossy compression (--lossy=80) and frame optimization (--optimize=3). Target: under 5MB for standard resolution.
5. **5. Metadata:** Embed a comment with the source URL and generation timestamp for traceability.

5.2 Output Presets

Preset	Resolution	Frames	FPS	Target Size
Thumbnail	480x300	8	4	< 1 MB
Standard	800x500	12	6	< 3 MB
High Quality	1200x750	15	8	< 8 MB
Mobile	375x667	10	5	< 2 MB

5.3 Encoding Commands

gifski (preferred)

```
gifski --fps 6 --quality 80 --width 800 \
  -o output.gif frame_*.png
```

FFmpeg fallback

```
# Step 1: Generate optimal palette
ffmpeg -i frame_%03d.png -vf \
  "fps=6,scale=800:-1:flags=lanczos,palettegen=stats_mode=diff" \
  palette.png
```

```
# Step 2: Encode with palette
```

```
ffmpeg -i frame_%03d.png -i palette.png -lavfi \
  "fps=6,scale=800:-1:flags=lanczos[x];[x][1:v]paletteuse=dither=bayer" \
  output.gif
```

gifsicle optimization

```
gifsicle --optimize=3 --lossy=80 --colors 256 \
  -o final.gif output.gif
```

5.4 Looping Strategy

The GIF should feel like a natural, continuous scroll preview. To achieve this:

- Capture frames scrolling down from top to maximum depth.
- Add a brief pause (2 duplicate frames) at the bottom-most position.
- Optionally: add reverse frames (scroll back up) for a ping-pong effect, or hard-cut back to the top with a slightly longer frame delay (500ms) to signal the loop reset.
- Set the GIF loop count to 0 (infinite loop).

6. Security & Abuse Prevention

> *Security is the highest priority. A headless browser that visits arbitrary URLs is a significant attack surface. Every layer must be hardened.*

6.1 SSRF (Server-Side Request Forgery) Prevention

This is the most critical security concern. Attackers may submit URLs that target internal infrastructure.

DNS Resolution Checks

- Resolve the URL's hostname to an IP address BEFORE passing it to Playwright.
- Block all private/reserved IP ranges: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, 127.0.0.0/8, 169.254.0.0/16, ::1, fc00::/7.
- Block cloud metadata endpoints: 169.254.169.254 (AWS), metadata.google.internal, etc.
- Re-validate after DNS resolution to catch DNS rebinding attacks (resolve, check, resolve again after 2s, check again).

URL Validation

- Only allow http:// and https:// schemes. Block file://, ftp://, data:, javascript:, and all others.
- Reject URLs with authentication info (user:pass@host).
- Reject IP-literal URLs (http://192.168.1.1) — force domain names only.
- Normalize and re-parse URLs to prevent encoding tricks (%00, unicode, etc.).

6.2 Rate Limiting

Tier	Limit	Window	Key
Anonymous IP	5 GIFs	1 hour	IP address
With CAPTCHA solve	15 GIFs	1 hour	IP + CAPTCHA token
Registered user	50 GIFs	1 hour	User ID
Global burst	100 concurrent jobs	—	System-wide

Implementation: Redis-based sliding window rate limiter. Return 429 Too Many Requests with a Retry-After header and remaining quota in response headers.

6.3 Domain Blocklist

Maintain a blocklist of domains that should never be crawled:

- Internal infrastructure domains (*.internal, *.local, *.corp).

- Government/military domains that prohibit automated access.
- Domains known to serve malware or illegal content.
- Adult content domains (maintain a curated list or use a classification API).
- Domains that have requested to be excluded (honor robots.txt Disallow for the screenshot user-agent).

> *Implement a community reporting mechanism. If a user reports a problematic GIF, add the source domain to a review queue.*

6.4 Browser Sandbox Hardening

- Run Chromium in a separate Docker container with minimal privileges (no-new-privileges, read-only root filesystem).
- Limit resource usage per container: max 512MB RAM, 1 CPU core, 30s execution timeout.
- Disable JavaScript execution selectively if the page is flagged as potentially malicious.
- Block all outbound network requests from the browser except to the target domain (network-level egress firewall).
- Disable file downloads, WebRTC, WebGL, and other browser features not needed for screenshots.
- Use a fresh browser context (incognito) for every job. No cookies, no storage, no cache carried between requests.

6.5 Content Safety

- After GIF generation, optionally run a content moderation API (e.g., Google Cloud Vision SafeSearch, AWS Rekognition) on the frames to detect NSFW or harmful content.
- Flag and quarantine GIFs that fail moderation. Do not serve them publicly.
- Implement a hash-based deduplication so known-bad content is blocked instantly on resubmission.

6.6 Input Sanitization Summary

Attack Vector	Mitigation
SSRF to internal services	DNS pre-resolution + IP range blocking + rebinding check
JavaScript injection via URL	Strict URL parsing, no javascript: scheme
Resource exhaustion (huge pages)	Max scroll depth, frame limit, execution timeout
Rapid-fire abuse	Multi-tier rate limiting (IP, user, global)
Malicious redirects	Max redirect chain depth (5), re-validate each hop
Data exfiltration via browser	Egress firewall, disable unnecessary APIs

Cryptomining via headless browser	CPU limits, execution timeout, process monitoring
Phishing/illegal content	Content moderation API, domain blocklist, user reporting

6.7 CAPTCHA Integration

Integrate a CAPTCHA (Cloudflare Turnstile recommended, as it is privacy-preserving and has a free tier) on the generation form:

- Show CAPTCHA after the first free generation, or when the rate limiter flags suspicious activity.
- Validate the CAPTCHA token server-side before enqueuing the job.
- CAPTCHA tokens are single-use and expire after 5 minutes.

7. Infrastructure & Deployment

7.1 Containerized Architecture

```
docker-compose.yml:
  services:
    web:        # Next.js frontend + API
    worker:     # BullMQ worker (scalable replicas)
    redis:      # Queue + rate limiting
    postgres:   # Job metadata
    browser:    # Playwright Chromium (sandboxed)
```

7.2 Deployment Options

Option	Frontend	Workers	Cost Tier
Vercel + Railway	Vercel Edge	Railway containers	$$
AWS (ECS/Fargate)	CloudFront + ALB	Fargate tasks	$$$
Fly.io (all-in-one)	Fly Machines	Fly Machines (auto-scale)	$$
Self-hosted VPS	Nginx + PM2	PM2 cluster	$

7.3 Storage & CDN

- Upload generated GIFs to S3/R2 with a 24-hour TTL lifecycle policy.
- Serve via CloudFront or Cloudflare CDN for fast global delivery.
- Use signed URLs for GIF access to prevent hot-linking abuse.
- Implement a cleanup cron job that deletes expired assets from both storage and database.

7.4 Scaling Strategy

- Frontend: stateless, horizontally scalable behind a load balancer.
- Workers: scale based on queue depth. Target: queue depth < 10 per worker.
- Browser instances: pool of 3–5 per worker, recycled every 50 jobs to prevent memory leaks.
- Redis: single instance with persistence (AOF) for rate limit continuity. Upgrade to Redis Cluster at scale.
- PostgreSQL: single primary is sufficient up to millions of jobs. Add read replicas if the status-polling load grows.

8. API Reference

8.1 POST /api/generate

Request Body

```
{
  "url": "https://example.com",
  "options": {
    "preset": "standard",
    "viewport": "desktop",
    "fps": 6,
    "maxFrames": 12
  },
  "captchaToken": "cf-turnstile-token-here"
}
```

Response (202 Accepted)

```
{
  "jobId": "a1b2c3d4-...",
  "estimatedTime": 18,
  "statusUrl": "/api/status/a1b2c3d4-..."
}
```

8.2 GET /api/status/:jobId

Response

```
{
  "jobId": "a1b2c3d4-...",
  "status": "PROCESSING",
  "progress": {
    "step": "capturing",
    "framesCaptures": 7,
    "totalFrames": 12,
    "percentComplete": 58
  }
}
```

8.3 GET /api/result/:jobId

Response (200 OK)

```
{
  "jobId": "a1b2c3d4-...",
  "status": "COMPLETED",
  "result": {
```

```
    "gifUrl": "https://cdn.example.com/gifs/a1b2c3d4.gif",
    "fileSize": 2340000,
    "dimensions": { "width": 800, "height": 500 },
    "frameCount": 12,
    "duration": 2.0
  },
  "expiresAt": "2026-03-22T14:30:00Z"
}
```

8.4 Error Response Format

```
{
  "error": {
    "code": "RATE_LIMIT_EXCEEDED",
    "message": "Too many requests. Try again in 12 minutes.",
    "retryAfter": 720
  }
}
```

8.5 Error Codes

Code	HTTP Status	Meaning
INVALID_URL	400	Malformed or unsupported URL
BLOCKED_DOMAIN	403	Domain is on the blocklist
RATE_LIMIT_EXCEEDED	429	Quota exhausted
SITE_UNREACHABLE	502	Target site did not respond
GENERATION_FAILED	500	Internal processing error
JOB_NOT_FOUND	404	Invalid or expired job ID
JOB_EXPIRED	410	GIF has been deleted (24h TTL)

9. Monitoring & Observability

9.1 Key Metrics to Track

Metric	Target	Alert Threshold
GIF generation p50 latency	< 15s	> 30s
GIF generation p99 latency	< 45s	> 60s
Job success rate	> 95%	< 90%
Queue depth	< 50	> 200
Worker CPU usage	< 70%	> 90%
Worker memory usage	< 400MB	> 480MB
Rate limit hits/min	< 50	> 200 (potential abuse)
Storage usage	—	> 80% capacity

9.2 Logging Strategy

- Structured JSON logs for all API requests (method, URL, status, latency, IP, jobId).
- Worker logs: job start/complete/fail with timing breakdowns (navigation, screenshot, encoding).
- Security events: rate limit hits, blocked domains, SSRF attempts, failed CAPTCHA validations.
- Ship logs to a centralized platform (Datadog, Grafana Loki, or CloudWatch).

9.3 Alerting

- PagerDuty/Slack alerts for: queue depth spikes, worker crashes, storage capacity warnings, error rate spikes.
- Weekly digest of: total GIFs generated, unique users, top domains, average file size, abuse attempts blocked.

— End of Document —